

Generic Programming is just Programming

Document #: P2692R0
Date: 2022-08-21
Project: Programming Language C++ SG17
Reply-to: Mihail Naydenov
<mihailnaydenov@gmail.com>

§ 1 Abstract

This paper argues, current generic programming strays too much from the promise of being like regular programming, to the point *correct* generic programming is an expert-only endeavor in what is essentially a new language.

§ 2 Motivation

“Generic Programming is just Programming”

B. Stroustrup

Good code is reusable code and reusable code is often parametrized. If the programmer wants to make a piece of code reusable by abstracting away the types, he/she has two options - “simple” or “advanced”:

Original code

```
void silly(MyClass& s)
{
    using type = MyClass::related_type;
    // ...
    if(s.has_something("hello"))
        s.do_it();
    // ...
}
```

Simple Generic

```
template<class S>
void silly(S& s)
{
    using type = S::related_type;
    // ...
    if(s.has_something("hello"))
```

```

    s.do_it();
    // ...
}

```

Advanced Generic

```

template<class S>
void silly(S& s)
{
    using char_type = MyExtensions::traits<S>::related_type;
    // ...
    if(MyExtensions::has_something(s, "hello"))
        MyExtensions::do_it(s);
    // ...
}

```

Let’s start with the good news: With the simple version, *we do* keep the promise, “generic programming is just programming”! With very little effort (and syntax for that matter) we turned our silly function into a generic function, one that will work with “any type”. In fact, this is *so good*, that it is, arguably, the de facto standard for 99% of the C++ developers. Every non-expert, non-library-writer most probably **will** write generic code in that manner.

It is all bad news from here on. As we know, the simple version code is not generic *enough*, in more aspects than one. To go “the extra mile”, make the code “right”, we have to use the advanced approach. Doing so however:

- A) We **must** considerably change how we write the function.
- B) We fall into a pit of severe problems and heavy compromises (P2279)¹.

While there are efforts to solve the pressing issues in **B**, it seems **we have accepted the loss of A**.

This proposal begs to disagree with that sentiment.

§ 3 Problems

§ 3.1 Dichotomy

First major problem with the status quo is the fact, we have *two* static polymorphism systems. As mentioned in **Motivation**, the user can choose b/w having it easy, but limited or having it powerful, but hard (developing effort, code changes, learning ramp). Because the second option does all the first one does, it is “better” and “more correct”, but with no natural progression (or discoverability!) b/w the two options there always be a pain-point in a project: “Is it worth it?”. This rises the question:

Is a project doing “simple templates” generic programming an “unprofessional” one?

(Assuming limitations in user experience like the need to use inheritance/wrapping, etc.)

This is an important question.

If the answer is YES, then the simple method is just for people who “don’t get it” or “don’t want to put the effort” (yet?).

If NO, then we have two systems intensionally for reasons like “legacy” and/or “simpler tasks” or something else, we convince ourselves into.

Either way, it is not a great place to be. In practice, most programmers will continue to use simple, “turn on templates” method and the “right” method will be limited to few “library writers”, who *really* have to cover all cases, and *really* want it to be easy for the end users.

There is also the question of **teachability**, including for people migrating from other languages.

How *exactly* one writes (static) generic code in C++ in the end? How can one port a library from another language, what are the language tools? Can an expert in, say, signal processing write good-to-great generic library, without being a C++ expert *as well*? And even if he/she is a C++ expert, will he/she *choose* “the right” way, considering barrier of entry for contributions?

§ 3.2 Explicit type mapping in the wrong place

Second serious problem is that this new method puts explicit re-mapping calls (of entities to types’s implementation) inside of the function body. This is *radically* different then most other generic programming, C++ or not. The other two most post popular methods, templates and dynamic polymorphism, both move this mapping outside of the function, delegating it to the class itself in one for or another.

The only other generic method that forces body code change for the purpose of remapping calls is “variant”, but that code change comes naturally as the author wants exactly that, to *remove* the mapping from the outside world, doing it manually inside the function instead. And even then, the author *can* use the objects as usual by manually selecting the type first:

Verbose variant usage

```
void silly(variant<A,S,D> t)
{
    // ...
    using V = variant<A,S,D>;
    variant_alternative_t<1, V>::type;
    // remap back to S
    get<1>(t).do_this();
    // remap back again to S
    get<1>(t).do_that();
    // remap back yet again to S
}
```

Simplified variant usage

```
void silly(variant<A,S,D> s)
{
    // ...
    auto s = get<1>(t); // remap just
                        // once to S
    decltype(s)::type;
    s.do_this();
    s.do_that();
}
```

In modern generic programming, however, we are forced to do the mapping on each call:

```

template<S_like T>
void odd(T t)
{
    // ...
    Sness::traits<T>::type; // remap back to S
    Sness::do_this(t);      // remap back again to S
    Sness::do_that(t);      // remap back yet again to S
}

```

What makes things infinitely worse is the fact, the mapping *itself* must first be created by the author! Literally, *a library* is needed for what the type does. This is quite different from *all* other generic programming, including in other languages, and different not in a good way. Having a piece of code that needs an accompanying library and then littering it with explicit calls to that machinery, replacing essentially trivial expressions like method calls, is a bad proposition. One will do it only if really has to.

This is not to say, the remapping itself is a bad thing! Quite the contrary, exactly this remapping is what makes the modern generic programming powerful, compared to the basic template. *How* (requiring a separate library) and *where* (invoked via explicit call each time, from inside the function) is unattractive, if not *weird*:

From the point of the user, the template argument *is* the customization point of the function/class. As far as the user is concerned, what the function/class does is customized on the type. Period. What we however mean behind “customization point” is an extra entity, which sits b/w the concrete type and the actual code. Combining these two, it turns out, we have ***customization point of a customization point!*** Seeing it from that perspective, it is no wonder, most people will not be bothered with the second layer of indirection.

§ 3.3 Syntax

Verbosity

By lacking singular mapping point for the entire function body people are forced to use explicit full paths into the library objects on each and every access to the objects. Contrasted to all the code elsewhere, generic and not, C++ or not, this is very jarring.

Forced free function call

Not all actions translate well to free functions calls. For some the fit is quite good if not perfect, for example `std::swap`, `execution::connect`², but for some, *arguably most*, moving to free function is completely unnatural (e.g. all getters and setters like `execution::get_scheduler`, `execution::set_value`, etc). Having properties of a type not being expressed as members will never sit well, especially when we have the same ones *already* as members like `get_allocator`.

Overall the syntax is “atypical” at best and quite certainly not something the vast majority of programmers (C++ or otherwise) will willingly opt for.

§ 4 Proposal

If we are to use the list from P2279, we should **add**.

— **non-obtrusive**

Where behind “obtrusive” we mean *both* verbosity and code style change. Ignoring the obtrusive nature of current state of affairs leads us, again, into developing *two* generic systems. No matter how much we polish the “advanced” one, if it still requires essentially a rewrite of the original algorithm into something “atypical”, chances are people will continue using the naive, simple to understand, simple to implement and simple to write approach. This is aggravated by the fact, there is a real chance we might not be able to fix all *other* issues of the current advanced model to the point “the style” being the only downside!

Where all this leaves us

Currently all of the C++ static polymorphism “customization points”, as described in P2279, are **obtrusive**. This is not “a fault” of the solutions, rather a particularity of the current language - one can’t “just” have a non-obtrusive solution, that is *also* uncompromisingly generic.

With that in mind, if “We need a *language* mechanism for customization points”, do we really want to actively pursue such a (language) solution that is *still* obtrusive? If we bite the bullet of changing the language, we better at least try to solve for “Generic Programming is just Programming”, in the hope of making it as easy and attractive as the “simple template” option. Otherwise the “simple template” will always be the “**Worse is Better**” alternative, if not standard.

-
1. We need a language mechanism for customization points: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2279r0.html>↔
 2. std::execution: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2300r1.html>↔